

Exploring the cosine similarity for automated relating of architectural issues and emails in mailing lists

Bjorn Pijnacker

Jesper van der Zwaag

Julian Pasveer

February 11, 2023

Abstract

In software development, it is important to document architectural design decisions. However, design decisions are often discussed in multiple places, such as in email lists or in issue tracking systems. The problem that arises is the fact that some decisions are hard to get a grasp on since a part of it could be explained in an email and part of it in an issue. In this research, we have investigated the relationships between design decisions documented in Jira issues and mailing lists for multiple Apache projects. Our research applied the cosine similarity measure in order to find relations between the issues and emails, followed by a qualitative and quantitative analysis that explored the relationships.

In general, we found that relationships are mostly one-way. A discussion most often starts with an issue and is followed up upon in an email. The most common relations were emails that were asking for discussions regarding an issue or emails that were referencing an issue. Moreover, emails were, in some cases, the cause of the creation of an issue, which was the only issue-to-email relation we encountered.

1 Introduction

The need to explicitly document design decisions in software has been emphasized both in research and in industry [1]. In order to make sure those design decisions are addressed, issue-tracking systems are used. Issue tracking systems, as the name suggests, are systems that track issues in software. An example of such a system is Atlassian Jira, in which bugs, tasks, change requests, and more can be managed [2, 3].

Software engineers often start a discussion on design decisions through emails in a mailing list and later continue the discussion in an issue submitted in an issue tracker. Other times, there is a discussion in a mailing list, after which an issue is created based on the discussion. However, these events can result in problems with understanding the design decisions and how they came to be, due to the scattered nature of the discussions between issues in issue trackers and emails in mailing lists.

The goal of this research is to find and investigate related architectural issues and emails that discuss design decisions. In order to achieve this, we aim to answer two research questions, which we will elaborate on in Section 3.1. We will apply the cosine similarity measure to find relations between the issues and emails, after which we will carry out both a qualitative and quantitative analysis in order to explore the types of relationships between architectural issues and emails.

In Section 2 we will discuss the background of architectural issues and mailing lists. We also discuss some other background information related to our research. Thereafter, in Section 3, we will elaborate on the study design regarding our research. In Section 4, we will review the results of our research and in Section 5 we will discuss said results. Next,

we state the threats to validity in Section 6, cover our conclusion in Section 7, and finally discuss future work in Section 8.

2 Background

2.1 Mailing lists, issue trackers, and architectural classification

In any software system, architecture plays a large role. The main components in determining such an architecture are architectural design decisions: decisions that are difficult or expensive to change at a later date. To get a good overview of the architectural decisions made in a project, these should be documented. This is, however, not always exhaustively done, leading to architectural knowledge evaporation [4]. As such, previous work has looked into the automatic classification of architectural design decisions in mailing lists [5] and issue tracking systems [6].

2.2 Cosine Similarity

The cosine similarity is an often-used similarity measure for two documents. In this measure, documents are represented as vectors. The vector for a document can be obtained by its term frequency, that is, the number of times a given word appears in that document. By assigning each word (over the documents to compare) a dimension in the vector, the two vectors can be compared.

The vectors obtained can be compared using their cosine. A value between 0 and 1 represents how much the vectors point in the same direction, and thereby, how similar the two documents are considered to be. The length of the documents does not directly influence the cosine similarity. As such, we can compare documents of wildly different sizes with each other.

2.2.1 TF-IDF Transformation

When a corpus of documents to compare has words that occur frequently while not indicative of a significant similarity, cosine similarity can produce results that are less than desired. An example, in this case, could be a stack trace. Stack traces often have many repeated words, but it is the words that are not frequent that make stack traces unique from one another. On the other side of the spectrum, specific names, e.g. a unique name of a class, are very likely to indicate a connection, but their occurrence would be overshadowed by the common words. To better check, if documents are similar, we want to reduce the impact of common words and amplify the impact of words that are distinctive.

We tackle this issue by using the TF-IDF transform. Using TF-IDF, we calculate the vector not by pure term frequency (tf) as explained above; we calculate it by multiplying the term frequency with the inverse document frequency, defined as:

$$\text{idf}(t, D) = \log \frac{N}{|\{d \in D : t \in d\}|},$$

that is, the logarithm of the number of documents N in the corpus divided by the number of documents D in the corpus in which the term t appears. This way, a word's importance is determined by its 'uniqueness' in the corpus: a word that occurs less often is deemed more significant to the text's meaning.

2.3 Stemming

Stemming is used for purposes of text normalization. From [7]: "The Porter stemming algorithm (or 'Porter stemmer') is a process for removing the commoner morphological and inflexional endings from words in English". It can be used to take multiple forms of

the same word (e.g. “creation” and “create”) and normalize them to an equal form (e.g. “creat”), for purposes of text comparison.

3 Study design

3.1 Research Questions

In our study we wish to answer the following questions:

RQ1 How can cosine similarity help us link related architectural issues and emails?

RQ2 Which types of relationships between architectural issues and emails can be demonstrated?

The goal of the first research question is to investigate how well cosine similarity works for linking architectural issues and emails. With the second research question, we wish to gain insights into how email lists are used in conjunction with issue trackers in the architectural design of a software product.

3.2 Datasets

For this research, we use two datasets: The first is a dataset of Jira issues belonging to multiple Apache projects: Cassandra, Hadoop, HDFS, MapReduce, Tajo, and Yarn [6]. The second is a set of emails from a mailing list for the same projects [8]. Each is obtained from previous work. We discuss each dataset below.

The dataset on Jira issues is presented in JSON format. Each entry contains information about the issue such as the title, body, comments, etc. A part of the issues is manually classified, as discussed in the report present in [6]. Of the approximately 70 thousand issues, 2174 have been classified by whether it discusses an architectural design decision. It is also specified whether this decision is an existence, executive, or property decision. Note that these are not mutually exclusive and that these were separate classifications.

The dataset of emails is classified using tags [8]. The full set of tags (obtained from the dataset) is shown in Table A.1. Lalis names in their thesis which accompanies the dataset that the architectural relevance of an email is determined by whether or not it has an architectural tag associated with it [9]. The exact set of architectural tags versus other non-architectural tags is not named, but we assume the layout as shown in Table A.1.

The dataset of Jira issues is presented in JSON format, while the dataset of emails is stored in H2 database format¹. For purposes of this research, we combined both into a PostgreSQL database of which the schema is shown in Figure 3.1. We show this schema purely for clarity to the reader, as it provides a high-level overview of the data used.

3.3 Research method

3.3.1 Similarity

To answer our first research questions, we will be implementing cosine similarity for the issues and emails to find related issues and emails. We will perform a number of comparisons on the issues and emails using the cosine similarity metric. These comparisons are the following:

- 1– All **architecturally relevant** emails are compared with *all* issues. We determine which emails are relevant by checking whether that email has tags associated with it as shown in Table A.1. We include only tags that are marked with a †, as others are not indicative of architecturally relevant design decisions [9]. This step concerns 70759 issues and 620 emails.

¹Note that we did not use any of the tools in [5] to browse the data, instead using the raw database

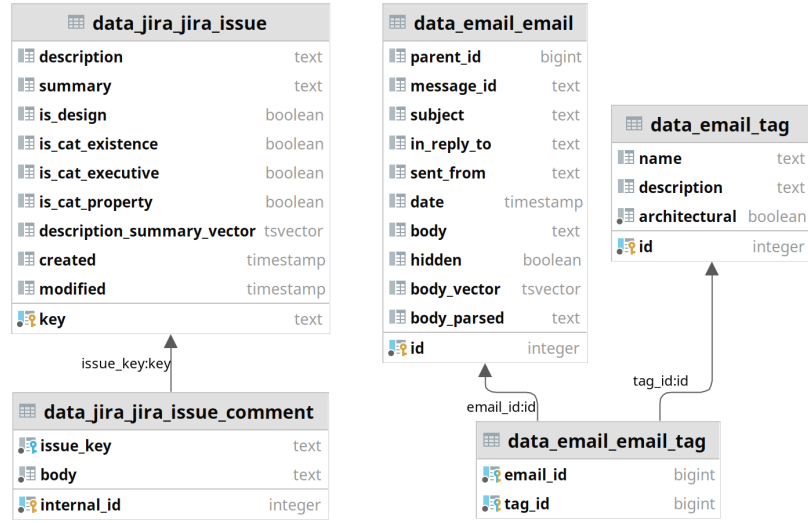


Figure 3.1: PostgreSQL database schema for combined datasets of [6, 8]

- 2- All **architecturally relevant** issues are compared with *all* emails. We determine which emails are relevant by checking whether `is_design` is true for that issue. This step concerns 1426 issues and 43815 emails.

Combining these two steps, we find a total of 106 350 770 comparisons.

Following the comparisons, the pairs are filtered to exclude too low similarities. We filter low similarities as this indicates no meaningful relationship and to reduce the scope of our manual work. To save space, we do not store similarities that are zero. As such, any further analysis concerns only pairs with a nonzero similarity (~ 30 million of 100 million).

We calculate the similarity of two given texts using two steps. Our first step for each text is to preprocess it. We perform stemming, and vectorization. Stemming will help us in comparing texts, as words with near-equal meanings, such as “create” and “creation”, are normalized to the same form. We then vectorize our text. To save us some work, we let our database do both stemming and vectorization for us, using the `to_tsvector` function that PostgreSQL provides. This function returns a list that contains each word and its position in the source text. Stopwords are filtered out and a form of stemming is applied automatically [10]. We do this as a pre-processing step for each issue summary concatenated with its description and for each email body. We do not consider email threads or issue comments.

The second step in the process involves determining the similarity between pairs of resulting texts by calculating their cosine similarity. In order to achieve this, the texts are transformed using the term frequency-inverse document frequency (TF-IDF) before the similarity computation. This approach was chosen as testing determined that the use of cosine similarity with raw word counts alone resulted in poor performance, as high similarities were often based on a single frequently occurring word, such as “test”, “junit”, or any word originating from a stack trace.

After the similarities have been calculated, we will attempt to find and classify the relations between pairs. This will help us answer RQ1 as it tells us how well the cosine similarity performed, and it will help us answer RQ2 by giving insight into which relation types exist.

3.3.2 Finding relationships

After performing the cosine similarity on the issues and emails we have a set of pairs that may or may not be related. As such, we wish to investigate the relationships that these pairs

may have. To obtain these relationships, we will be using the grounded theory method.

Grounded theory is a qualitative research method that aims to iteratively generate a theory from data. It is a type of research methodology that focuses on the process of discovering or “grounding” theory from data that has been collected. The goal of grounded theory is to develop a theory that is “grounded” in the data, meaning that it is derived from and supported by the data. Instead of (dis)proving a possible theory, or a hypothesis, we go through the data step by step and continuously alternate between collecting data and analyzing it. With the gradual analysis, the extracted theory increasingly gets more plausible. In order to find relationships between architectural emails and issues, we conduct a qualitative analysis using the grounded theory methodology. We start off by constructing a protocol, a set of guidelines determining the grounded theory process, to which we adhere.

We first determine the initial research questions. These can be found in Section 3.1. Our main research question for this second part of our research is “Which types of relationships between architectural issues and emails can be demonstrated”. We then recruit and collect the data that we will be using. This is what we have done in Section 3.3.1. The collected data will be analyzed by using an iterative process, the property of grounded theory that makes it stand out from other approaches. The process starts by breaking data into excerpts, also known as coding. During this procedure, we attach codes to indicators. Coding, in this case, refers to labeling data, whereas indicators are the actual data, also known as incidents.

After open coding has been completed, we move on to axial coding, in which the indicators are grouped according to assigned codes. These groupings will then be examined in order to find concepts and categories. The coded data will be iteratively reviewed to refine emerging concepts. The final step is the so-called sense-making step, in which we write a field memo. This concludes the grounded theory.

As mentioned in Section 3.3.1, the data is filtered using a similarity algorithm. We consider the pairs of issues and emails with a similarity above 0.35 for our grounded theory-analysis. The research questions have already been determined, and the data has already been collected and is at our disposal. Below, we go into more detail about our exact process of open coding, axial coding, and sense-making.

Open Coding We split the data set into three equally sized parts. Each author codes one of these parts, discussing with other authors in case of uncertainties. These results are then discussed between all authors in order to obtain a preliminary set of axial codes.

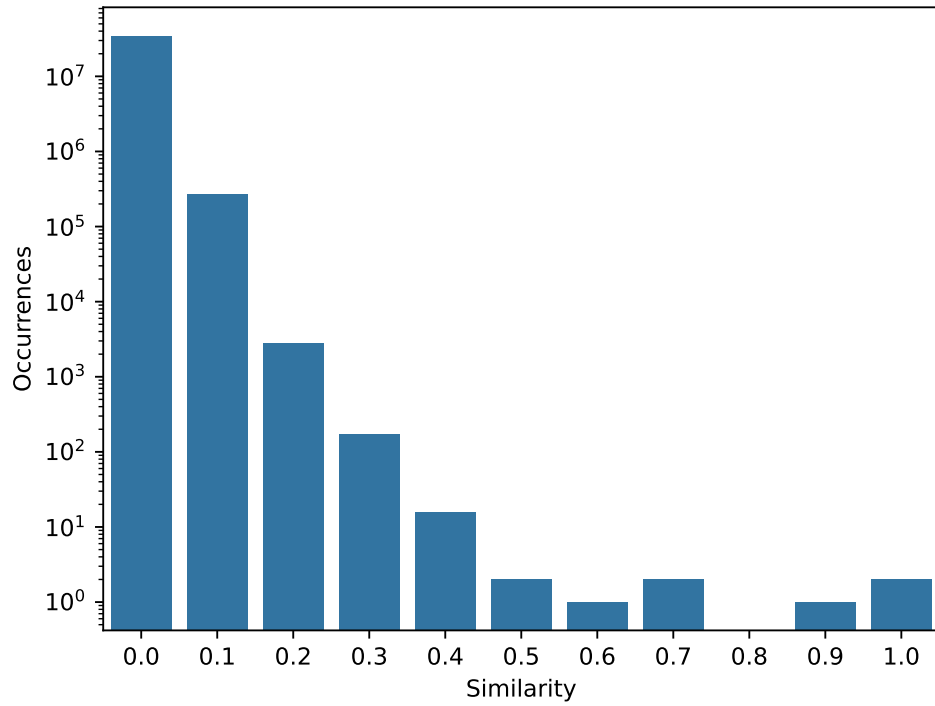
Axial Coding After completing open coding, we discuss with all authors and determine an initial set of labels to use for axial coding. Each email is tagged with one or more of these labels. We performed this step iteratively and discussed it with all authors until we found an agreement on a suitable set of labels.

4 Results

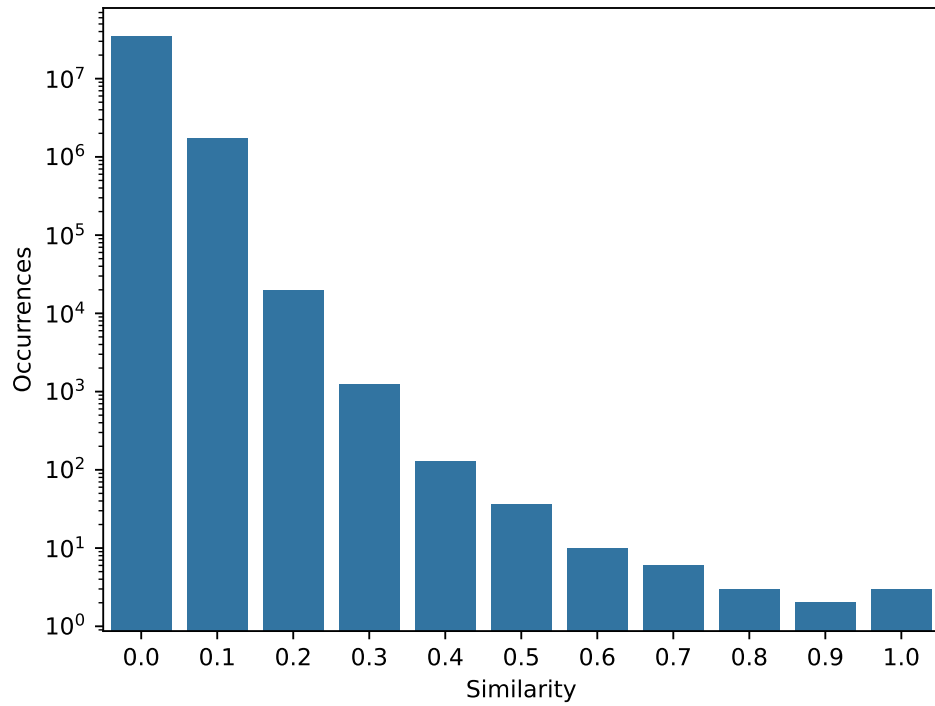
In Figure 4.1 we show similarity counts of pairs after cosine similarity. This figure clearly indicates that there is quite a low number of pairs with significant similarity. Based on these results we chose our cut-off of 0.35 to do further analysis.

In Table 4.1 we show all labels that we found using axial coding.

In Figure 4.2 and Figure 4.3 we show the occurrences of each axial code and the occurrences of each axial code against similarities respectively.

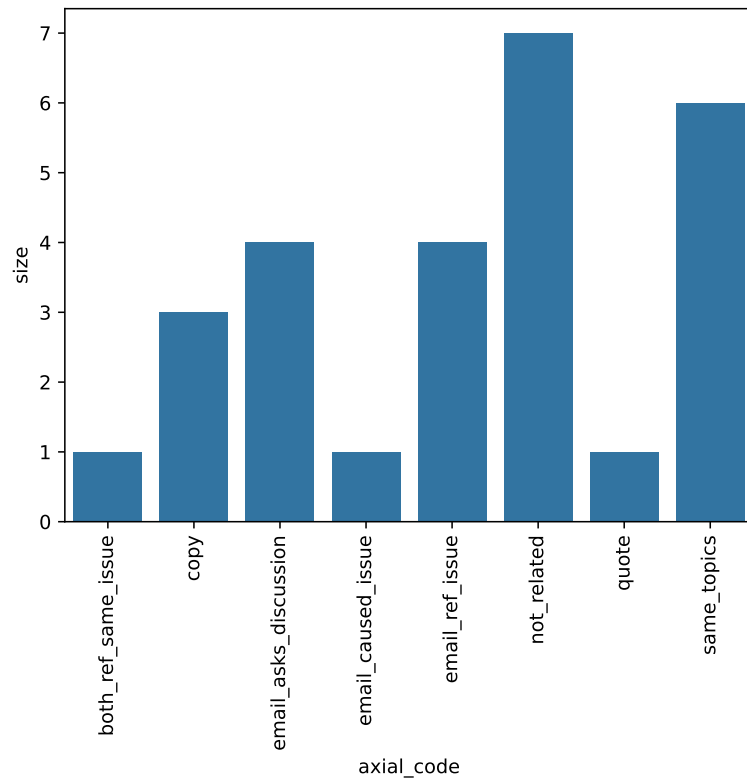


(a) Similarity groups for architectural emails compared to all issues.

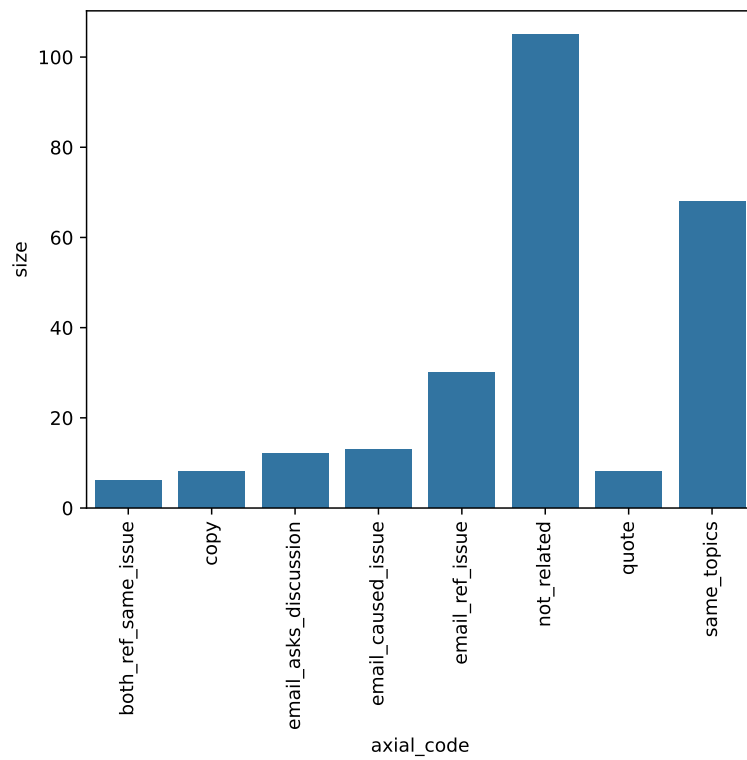


(b) Similarity groups for architectural issues compared to all emails.

Figure 4.1: Similarities of found pairs. Note that any pair with a similarity of exactly zero is excluded

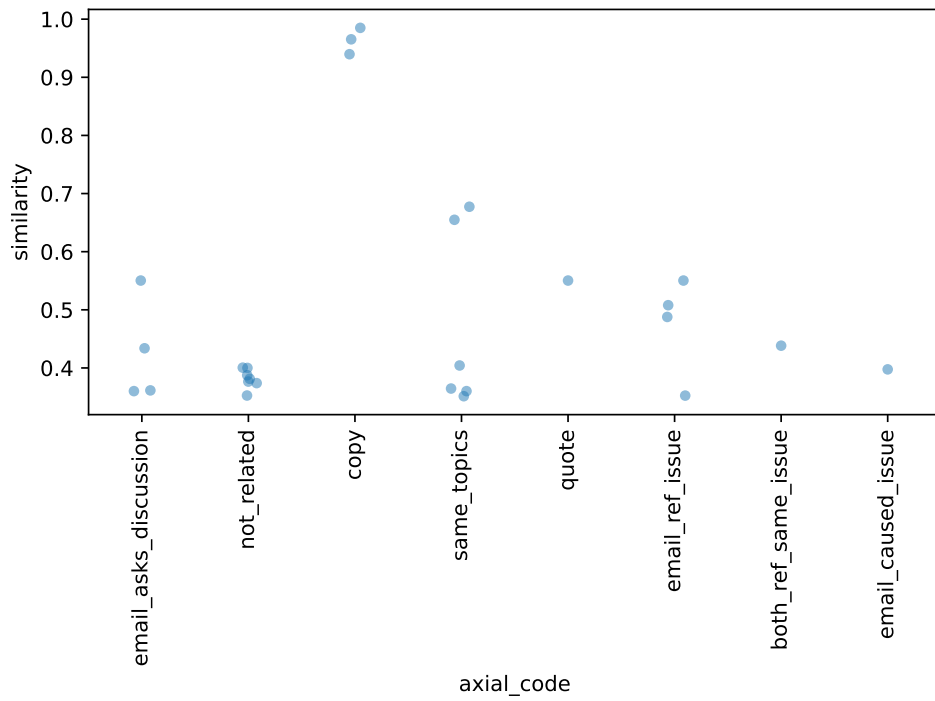


(a) Axial coding results for architectural emails compared to all issues

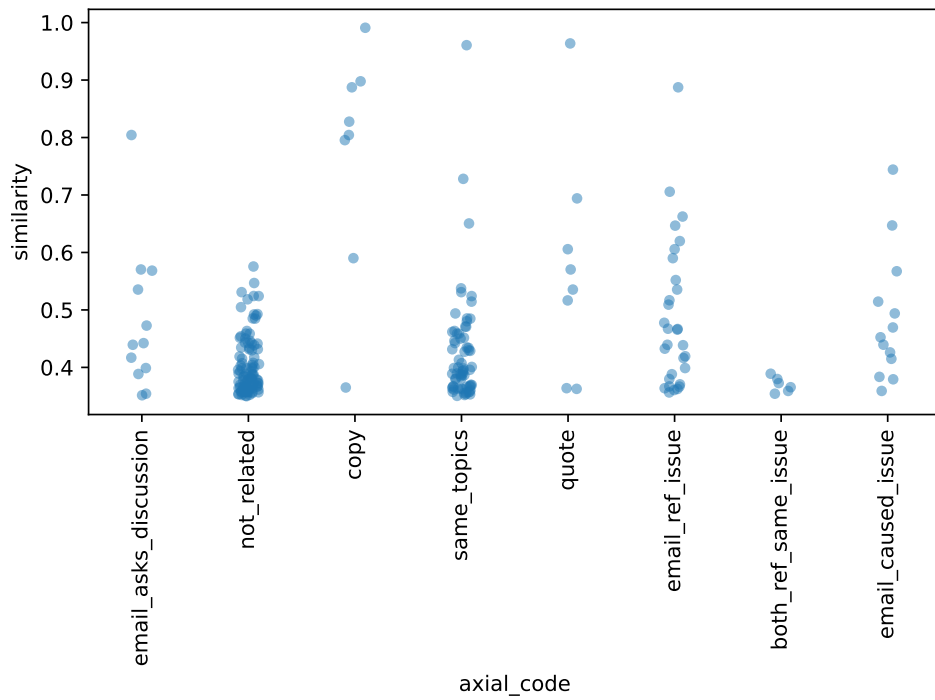


(b) Axial coding results for architectural issues compared to all emails

Figure 4.2: Occurrences of each axial code for both comparison directions



(a) Axial coding results per similarity bucket for architectural emails compared to all issues



(b) Axial coding results per similarity bucket for architectural issues compared to all emails

Figure 4.3: Occurrences of each axial code per similarity for both comparison directions

Name	Explanation
not_related	The issue and the email do not have any specific relation between them.
quote	The email quotes the issue partially, or vice versa.
email_ref_issue	The email references the issue by name.
copy	The email quotes the issue completely, or vice versa.
email_asks_discussion	The email asks readers to discuss or review an issue.
same_topics	The email and issue discuss similar topics. Often used in conjunction with not_related.
email_caused_issue	The email discusses something that results in an issue being created.
both_ref_same_issue	Both the email and issue reference the same issue.

Table 4.1: Labels used for axial coding. Note that the last three labels were added in a later iteration of axial coding. We consider all tags except not_related and same_topics to be indicative of a meaningful relation between email and issue.

5 Discussion

For the comparison of architectural issues with all emails, of 188 pairs, 65 (34,6%) were determined to be meaningful relationships. For the comparison of architectural emails with all issues, of 24 pairs, 12 (50%) were determined to contain a meaningful relationship between the email and the issue. We also see this in Figure 4.2.

In Table 4.1 we show all labels that we found using axial coding. From these labels, we see immediately that relationships are mostly one-way: emails reference or discuss issues, and issues may reference other issues, but no emails directly reference other emails, and no issues reference emails. The only relationship between emails to issues is the creation of an issue based on an email.

In Figure 4.3 we see the found relationships compared with the similarity buckets of the issue-email pair in question. We see that the most common meaningful code is ‘email asks discussion’ and ‘email ref issue’, and we can see that in both directions, all more complex relations showed up between similarity of 0.4 and 0.7 and that above 0.7, we mostly see ‘copy’ or ‘quote’. This makes sense, as more words in common means a higher similarity.

It is clear that in most cases the relations are emails that discuss issues. The protocol of introducing changes into a software product does involve the creation of an issue to track the changes. Often, we require discussion around changes, though not everybody may see the issue. To reach out to fellow developers, or to users, email is an easier and more widespread way to attain this goal. Moreso, we see that emails reference issues, but not vice versa. This too makes sense: an issue has an easy-to-use unique key that can be referenced while referencing an email is simply more difficult.

Also notable is the difference in the number of occurrences between the figures in Figure 4.1. With the cosine similarity, we were only able to find 24 issues with a similarity higher than 0.35 to the architectural emails, while we found 188 emails with a similarity higher than 0.35 to the architectural issues.

It is possible that architectural emails have relations with fewer issues than architectural issues have with emails, meaning that architectural decisions taken in emails don’t need a corresponding issue in the same way that an architectural issue has related emails, but future work is required to look into this specific phenomenon in depth.

6 Threats to Validity

In our work, we have three main threats to validity. We discuss these below.

The first is concerned with the Jira issues dataset. We assumed that all issues were classified to either be or not be architectural. However, only a subset (approximately 2000) of the issues was classified. This means that, in our comparisons, many issues may have been skipped that contained meaningful insights for our research. This discrepancy in the number of classified issues occurred because we pulled all issues from the Jira projects as described in [6]. The authors of [6] helped us with this, but they named that they were unable to classify the new set of issues completely. They named that the classification would take so long that this research would not be compatible in due time.

The second threat to validity is the low amount of data we worked with in combination with the usage of the cosine similarity. Of approximately 100 million comparisons, only 212 were deemed significantly similar by the cosine similarity. This is due to the fact that cosine similarity does not take semantics into account. Certain words may be spelled the same way, but can semantically have very different meanings. Furthermore, synonyms are also not taken into account which results in the reduction of possible matched pairs. Pairs that were not deemed similar may have contained significant relationships that we did not find in this research. This could potentially be mitigated by investigating all pairs for a meaningful relation, not only those that are similar. To reduce this to a usable scope, one

could perform a random sample of all the data and investigate this, something which we did not do. Moreover, using a different method instead of cosine similarity could result in finding more pairs, since things like semantics can be considered.

A third threat results from the grounded theory-protocol. The set of issues to analyze was split in three: one-third for each author, without overlap. This means that some coding may be incorrect, as the combination of analyses may have resulted in a different coding if each issue had been coded by multiple authors.

7 Conclusion

In this paper, we explored the use of cosine similarity for relating mailing list emails and issues from an issue tracker for the same software project, in the hopes of being able to automate finding related documents and gain insight into the relationships that appear. We compared architectural issues with the full set of emails, and we compared architectural emails with the full set of issues. The architectural classification of issues and emails is part of previous works [6, 9].

Of over 100 million comparisons, we analyzed 212 pairs with cosine similarity over 0.35. We found that of these pairs, 34.6% (meaningful relationships between the issue and the email) and up to 50% (meaningful relationships between the email and the issue) of email-issue pairs were shown to have a meaningful relationship.

We also determined that multiple types of relations between emails and issues exist, mostly one-way: from email to issue. The most common relations are an email asking for discussion regarding an issue, or an email referencing an issue. We also observed emails causing the creation of an issue: the only issue-to-email relation we found.

To us, it made sense that relations are mostly emails discussing issues. Discussions frequently talk about changes, but the problem is that not everyone necessarily sees the issue. An email is more convenient as a means to communicate with others, and therefore reference issues more often than the other way around.

8 Future

As discussed in Section 6, the cosine similarity had fewer results than we expected to find. We do not believe a set of emails and issues of 40–70 thousand entries each has only 212 document pairs that are related. Moreso, these results were only up to 50% actually meaningful. Future work should test different similarity measures and see whether these perform better. A start may be the use of a semantic similarity such as word2vec, which assigns similar vectors to documents with a semantically similar meaning, instead of just documents with similar words, as the cosine similarity does.

Future work should also consider the not-similar issue-email pairs for the ground theory process. While we only considered the pairs with notably high similarity, there could be relations in the not-similar pairs that we did not expose.

In our work, we do not consider email threads as a single entity, and we do not consider issue comments. These sources of discussion may prove useful in finding similar documents; however, we skipped this to save time during the similarity calculation and subsequent coding. Future work may wish to explore this data further, and see if useful conclusions can be drawn from it, or if different relation types may appear.

As noted in Section 5, we found a large difference in the number of occurrences between the figures in Figure 4.1. To investigate whether it is true that architectural decisions taken in emails have a lower number of issues related to it than vice versa, future work is required. This future work should investigate architectural decisions in both emails and issues, and

see whether these are different in nature. This will give insight into the use of these communication channels and the type of architectural correspondence in each.

Lastly, our, but also future approaches should be tested on different projects than only Apache projects. Apache, like any software project, has a certain protocol for doing work and proposing changes, which may not extend to other projects. A good starting point may be to extend our approach to a general set of Agile projects.

References

- [1] M. Bhat, K. Shumaiev, A. Biesdorf, U. Hohenstein, and F. Matthes, “Automatic Extraction of Design Decisions from Issue Management Systems: A Machine Learning Based Approach,” in *Software Architecture*, A. Lopes and R. de Lemos, Eds., Cham: Springer International Publishing, 2017, pp. 138–154, ISBN: 978-3-319-65831-5.
- [2] M. Soliman, M. Galster, and P. Avgeriou, “An Exploratory Study on Architectural Knowledge in Issue Tracking Systems,” in *Software Architecture*, S. Biffl, E. Navarro, W. Löwe, M. Sirjani, R. Mirandola, and D. Weyns, Eds., Cham: Springer International Publishing, 2021, pp. 117–133, ISBN: 978-3-030-86044-8.
- [3] Atlassian. “Jira Software - Features,” Atlassian. (), [Online]. Available: <https://www.atlassian.com/software/jira/features> (visited on 01/29/2023).
- [4] A. Jansen and J. Bosch, “Software Architecture as a Set of Architectural Design Decisions,” in *5th Working IEEE/IFIP Conference on Software Architecture (WICSA’05)*, Nov. 2005, pp. 109–120. DOI: 10.1109/WICSA.2005.61.
- [5] A. Lalis, “ArchitecturalKnowledgeAnalysis,” GitHub. (), [Online]. Available: <https://github.com/ArchitecturalKnowledgeAnalysis> (visited on 01/06/2023).
- [6] J. Maarleveld and A. Dekker, *Mining-design-decisions*, mining-design-decisions, Sep. 1, 2022. [Online]. Available: <https://github.com/mining-design-decisions/mining-design-decisions> (visited on 01/06/2023).
- [7] “Porter Stemming Algorithm.” (), [Online]. Available: <https://tartarus.org/martin/PorterStemmer/> (visited on 02/04/2023).
- [8] A. Lalis, *Datasets*, ArchitecturalKnowledgeAnalysis, Aug. 16, 2022. [Online]. Available: <https://github.com/ArchitecturalKnowledgeAnalysis/Datasets> (visited on 01/06/2023).
- [9] A. Lalis, “Searching for Architectural Design Decisions in Open-Source Software Mailing Lists,” BA thesis, University of Groningen, Groningen, the Netherlands, Aug. 23, 2022, 72 pp. [Online]. Available: <https://github.com/ArchitecturalKnowledgeAnalysis/EmailSearchEfficacyThesis/blob/main/thesis.pdf> (visited on 01/06/2023).
- [10] PostgreSQL Global Development Group. “12.3. Controlling Text Search,” PostgreSQL Documentation. (Nov. 10, 2022), [Online]. Available: <https://www.postgresql.org/docs/15/textsearch-controls.html> (visited on 01/06/2023).

A Email tags

Table A.1: Tags that may be applied to an email [8]

Name	Description
existence-behavioral [†]	More specific version of the "existence" tag for differentiating emails that focus on the behavior of a system.
existence [†]	Used for discussions about the structure or behavior of the system, like discussing the addition, removal, or updating of internal components, or how those components interact. It can be thought of as making assertions and decisions about the software's own internal structure and behavior. This is likely to be the most common tag that's used when dealing with architectural emails.
existence-structural [†]	More specific version of the "existence" tag for differentiating emails that focus on the structure of a system, like the components that make it up.
technology [†]	Used to tag discussions about external technologies that may or may not be used in the software.
not-ak	Utility tag that's used to indicate that an email has been read, and does not contain any architectural knowledge. Usually, this is used when an email cannot be categorized using any of the other tags.
conflict	Utility tag that's used to indicate a conflict between the authors of emails in a thread.
process [†]	The tagged email should discuss decisions or opinions on the development process. This includes things like best practices, testing strategies, release strategies, code organization, and communication practices.
announce-system	Used when an email is sent to announce the creation or intent to create a new system within a software architecture.
requirements-analysis	Utility tag that's used for labeling emails that analyze the requirements of all or part of the software. Will probably appear in conjunction with the "new-architecture" and "announce-system" tags quite often.
updating-architecture	Utility tag that's used when updating the architecture of a system. Usually used for large emails with big plans about major refactorings or updates.
compare-solutions	Utility tag that's used when authors compare different solutions to a problem being discussed.
new-architecture	Used when authors discuss an entirely new architecture within a project.
property [†]	Used to tag emails that contain discussion or a decision about an abstract property of the system. For example, making a claim that the system must meet some set of requirements, or that it must support integration with some services.

Continued on next page

Table A.1: Tags that may be applied to an email [8] (Continued)

ban	Used when someone explicitly decides to not do something, like: to not include a technology; to not use a proposed structure; to not support some set of requirements
attached-info	Used to mark emails that have external links or files to extra information which might be of use. Note that we are purely interested in email content, so external content does not influence the tags applied to an email.
duplicate	Used in conjunction with “not-ak” to indicate that the reason for tagging an email as “not-ak” is because it is a duplicate of another email.
WIP	Apply this tag to any emails as an indicator that their analysis is not yet complete.
funny	Used for marking particularly funny content in emails. Not really useful from a scientific perspective though.

[†] Emails annotated with these tags are architecturally relevant [9]